

Zoo Animal Records

Overview

In this in-lab assignment, you will be asked to maintain records of animals in a zoo. You must use the given header file (without editing it) and complete all the required functions in a .cpp file to implement the required functionality. This assignment will test your ability to create classes and objects, abstract classes, inheritance, virtual functions, etc. The database of animals must be maintained in a binary search tree that you must implement (using the header file that is given). The binary search tree will allow us to answer slightly richer forms of query (than last week). Now, we will be able to output a range of animals.

Animals

In general, animals can be divided into several types like birds, mammals, reptiles, etc. Each individual animal will be a specific species under an appropriate type. For example, a parrot named polly will be an animal of type bird and of species parrot. For the purpose of this assignment, we will have two types of animals: Birds and Mammals. Under each type we have the following species.

- Birds
 - peacock
 - parrot
- Mammals
 - tiger
 - monkey

Note that if the species is peacock or parrot, the type bird should be inferred automatically. Likewise, if the species is tiger or monkey, the type mammal should be inferred.

Every animal has a weight associated with it.

Given

You will be given a header file. This file contains several class

`zoo-week02.h`

declarations and some function definitions. Carefully go through the header file to see what classes are given, which ones are inherited, etc.

The given header file should NOT be modified.

Input/Output Format

Each line in the input can either be a *birth* record that creates a new animal, a *death* record that deletes an animal from the records, or a *query*. Each birth is of the form:

```
birth <species-type> <name> weight
```

For example, `birth parrot polly 1.5` should add a parrot named polly and record its weight as 1.5 KGs.

A death record is easier. It is just `death <name>`. That should just delete the animal

with the given name. For example, `death polly` unfortunately records the death of the animal named polly. If there is no animal named polly, the list is unchanged.

A query takes two numbers a and b (type double with $a \leq b$) and prints the number of animals whose weight is between a and b . Additionally, there can be an optional parameter that takes a species-type. When the species-type is specified, the query must only return the number of animals of the given species-type whose weight is between a and b .

```
query <a> <b> <optional species-type> end-query
```

For example,

- the input `query 2.5 5 bird end-query` must print the number of birds whose weight is between 2.5 and 5 KGs, whereas
- the input `query 5.0 50.0 end-query` must print the number of animals (of all types) between 5 and 50 KGs.

